Agent Governance Implementation Guide

Add governance to the agent you already built.
CrewAI. LangGraph. Pydantic AI. Step by step.

EVOKED.DEV

Version 1.0 · 2026

Agent Governance Implementation Guide

Agent Governance Implementation Guide

Add governance to the agent you already built. Framework-specific. Step by step.

Chapter 1: Run the Scan

Before you build anything, measure what you have.

```
npx agent-governance-check
```

This scans your project for five governance properties. Each one comes back FOUND, PARTIAL, or MISSING. Your score tells you where to start.

Priority order for implementation:

1. Identity first - everything else anchors to this
2. Refusal second - trust requires boundaries
3. Memory third - accountability needs a record
4. Deliberation fourth - multi-agent systems need conflict resolution
5. Drift last - you need baselines before you can measure change

If you scored 0/5, start with identity and refusal. Those two properties change the nature of your system. The rest build on them.

Chapter 2: The Five Properties

Property 1: Persistent Identity

An agent without identity drifts. Its behavior changes as prompts evolve, models update, and context shifts. Nobody notices until something breaks.

Identity is not a system prompt. A system prompt is configuration - it changes with every deploy. Identity is a commitment - it persists across changes and gives you something to measure against.

What identity looks like in practice:

- A charter document that defines purpose, boundaries, and commitments
- A role definition that survives model updates
- Constraints that the agent holds even when they cost performance

Property 2: Refusal Capability

An agent that can't say no can't be trusted. Guardrails filter inputs. Refusal is the agent's own judgment about what it should not do.

Six refusal categories:

Category	The Agent Says	Example
Scope	"That's outside my expertise"	Recipe agent refusing financial advice
Safety	"That could cause harm"	Nutrition agent refusing eating disorder triggers
Consent	"I don't have permission"	Agent refusing to share user data without consent
Integrity	"That would be dishonest"	Agent refusing to fabricate sources or

Category	The Agent Says	Example
		citations
Energy	“That wastes resources”	Agent refusing infinite retry loops
Legal	“That violates regulations”	Agent refusing GDPR-violating data processing

For each category: define the trigger, the refusal behavior, and how to test it.

Property 3: Decision Memory

Agents without memory make the same mistakes forever. They can’t explain their past decisions. They can’t be audited.

What to persist:

- Decisions made and reasoning
- Refusals triggered and why
- User interactions that changed behavior
- Errors and how they were handled

What to expire:

- Session-specific context after session ends
- Cached intermediate results
- Temporary state that doesn’t inform future behavior

Property 4: Disagreement Protocol

When agents receive conflicting instructions, what happens? When a human overrides an agent, is that recorded? When two agents disagree, who arbitrates?

Even single-agent systems need a disagreement protocol. The disagreement is between the agent and the human who gave it conflicting instructions.

Property 5: Drift Detection

You need baselines before you can measure change. Define what "normal" looks like for your agent, then measure against it.

Five dimensions to track:

1. Voice consistency - does the agent still sound like itself?
2. Principle adherence - does behavior match stated commitments?
3. Engagement quality - are interactions genuine or mechanical?
4. Energy sustainability - is the system maintainable?
5. Values alignment - does output reflect stated values?

Chapter 3: CrewAI Implementation

CrewAI uses YAML configuration for agents and tasks. Governance layers onto this naturally.

Adding Identity

In `agents.yaml`, the `role` and `backstory` fields are your identity layer:

```yaml
# agents.yaml
research_analyst:
  role: "Governance-Aware Research Analyst"
  backstory: >
    You are a research analyst who operates under a governance charter.
    Your purpose is to find accurate, verifiable information.
    You refuse to fabricate sources, speculate beyond evidence,
    or process requests outside your research domain.
    When uncertain, you say so. When asked to cross a boundary, you decline.
  goal: "Provide accurate, well-sourced research with clear limitations"
  verbose: true
```

Store your full charter in a separate file and reference it:

```python
# crew.py
from pathlib import Path

charter = Path("governance/agent-charter.md").read_text()

crew = Crew(
    agents=[research_analyst],
    tasks=[research_task],
    verbose=True,
    # Pass charter as shared context
    manager_agent=Agent(
        role="Governance Monitor",
        backstory=charter,
        goal="Ensure all agents operate within charter boundaries"
    )
)
```

Implementing Refusal

Custom tools are where refusal lives in CrewAI:

```python
from crewai.tools import tool

REFUSAL_TRIGGERS = {
    "scope": ["financial advice", "medical diagnosis", "legal counsel"],
    "safety": ["eating disorder", "self-harm", "dangerous activity"],
    "consent": ["share user data", "export personal", "send to third party"],
}

def check_refusal(query: str) -> str | None:
    """Check if a query triggers a refusal category."""
    query_lower = query.lower()
    for category, triggers in REFUSAL_TRIGGERS.items():
        for trigger in triggers:
            if trigger in query_lower:
                return f"REFUSED [{category}]: Request contains '{trigger}'. This is
        outside my restraint boundaries."
    return None

@tool
def search_database(query: str) -> str:
    """Search the research database with governance guardrails."""
    refusal = check_refusal(query)
    if refusal:
```

```
        return refusal
    return execute_search(query)
```

Decision Logging

Add a callback that logs decisions:

```
 from crewai.utilities import Logger
import json
from datetime import datetime

class GovernanceLogger:
    def __init__(self, log_path="governance/decision-log.jsonl"):
        self.log_path = log_path

    def log_decision(self, agent_role, task, result, refusals=None):
        entry = {
            "timestamp": datetime.utcnow().isoformat(),
            "agent": agent_role,
            "task": task,
            "result_summary": result[:200],
            "refusals": refusals or [],
        }
        with open(self.log_path, "a") as f:
            f.write(json.dumps(entry) + "\n")
```

Chapter 4: LangGraph Implementation

LangGraph's state-based architecture makes governance a natural graph node.

State Schema with Governance

```
 from typing import TypedDict, Annotated, Literal
from langgraph.graph import add_messages

class GovernedState(TypedDict):
    messages: Annotated[list, add_messages]
    agent_id: str
```

```
    session_decisions: list[dict]
    refusal_log: list[dict]
    governance_status: Literal["pass", "refuse", "escalate"]
    charter_hash: str  # Detect if charter changes mid-session
```

Governance Nodes

```
 from langgraph.graph import StateGraph, END

def governance_check(state: GovernedState) -> GovernedState:
    """Check the latest message against refusal categories."""
    latest = state["messages"][-1].content
    refusal = check_refusals(latest)

    if refusal:
        state["refusal_log"].append({
            "message": latest[:100],
            "category": refusal.category,
            "reason": refusal.reason,
        })
        state["governance_status"] = "refuse"
    else:
        state["governance_status"] = "pass"
    return state

def log_decision(state: GovernedState) -> GovernedState:
    """Log the agent's decision after execution."""
    state["session_decisions"].append({
        "input": state["messages"][-2].content[:100],
        "output": state["messages"][-1].content[:200],
        "agent_id": state["agent_id"],
    })
    return state

# Build the graph
graph = StateGraph(GovernedState)
graph.add_node("governance_check", governance_check)
graph.add_node("agent", agent_node)
graph.add_node("log_decision", log_decision)

graph.set_entry_point("governance_check")
graph.add_conditional_edges(
    "governance_check",
    lambda s: s["governance_status"],
    {"pass": "agent", "refuse": "refusal_response", "escalate": END}
```

```
)
graph.add_edge("agent", "log_decision")
graph.add_edge("log_decision", END)
```

Conditional Routing Through Governance

The key pattern: every message passes through the governance check node before reaching the agent. Refusals short-circuit. Passes proceed. Escalations halt.

```
Entry -> governance_check -> [pass] -> agent -> log_decision -> END
                          -> [refuse] -> refusal_response -> END
                          -> [escalate] -> END (human review)
```

Chapter 5: Pydantic AI Implementation

Pydantic AI's structured approach makes governance a type-safe concern.

Agent with Structured Identity

```
from pydantic_ai import Agent
from pydantic import BaseModel
from typing import Literal
from pathlib import Path

class GovernedResponse(BaseModel):
    answer: str
    decision_reasoning: str
    refusals: list[str] = []
    governance_status: Literal["pass", "refuse", "escalate"]

charter_text = Path("governance/agent-charter.md").read_text()

agent = Agent(
    "anthropic:claude-sonnet-4-20250514",
    system_prompt=f"""You operate under the following governance charter:

{charter_text}
```

```
For every response:
1. Check if the request falls within your charter boundaries
2. If it does, respond with governance_status "pass"
3. If it doesn't, set governance_status to "refuse" and explain in refusals
4. If you're uncertain, set governance_status to "escalate"
5. Always include decision_reasoning explaining your governance judgment
""",
    result_type=GovernedResponse,
)
```

Tool-Level Refusal

```
from pydantic_ai import RunContext

@agent.tool
async def search_records(ctx: RunContext, query: str) -> str:
    """Search records with governance boundaries."""
    # Scope refusal
    if is_out_of_scope(query):
        return "REFUSED [scope]: This query is outside my domain expertise."

    # Consent refusal
    if contains_pii_request(query) and not ctx.deps.user_has_consent:
        return "REFUSED [consent]: Cannot process personal data without explicit
        consent."

    return await ctx.deps.db.search(query)
```

Decision Logging with Result Validators

```
@agent.result_validator
async def log_and_validate(ctx: RunContext, result: GovernedResponse) ->
        GovernedResponse:
    """Log every decision and validate governance compliance."""
    log_entry = {
        "timestamp": datetime.utcnow().isoformat(),
        "query": ctx.prompt[:200],
        "status": result.governance_status,
        "reasoning": result.decision_reasoning,
        "refusals": result.refusals,
    }
    await ctx.deps.decision_log.append(log_entry)
```

```
    # Validate: refusals should have governance_status "refuse"
    if result.refusals and result.governance_status == "pass":
        result.governance_status = "refuse"

    return result
```

Chapter 6: Building Refusal in Detail

For each refusal category, here is the pattern: the trigger, the behavior, and the test.

Scope Refusal

Trigger: Request falls outside the agent's defined domain.

```
DOMAIN_KEYWORDS = ["recipe", "meal", "ingredient", "cooking", "food"]

def is_out_of_scope(query: str) -> bool:
    query_lower = query.lower()
    return not any(kw in query_lower for kw in DOMAIN_KEYWORDS)
```

Behavior: Return a clear, non-judgmental refusal.

```
"I'm designed to help with meal planning and recipes. This question is outside my
        area - you'll get a better answer from a specialist."
```

Test:

```
def test_scope_refusal():
    result = agent.run_sync("What stocks should I invest in?")
    assert result.data.governance_status == "refuse"
    assert "scope" in result.data.refusals[0].lower()
```

Safety Refusal

Trigger: Request could cause psychological or physical harm.

```
SAFETY_TERMS = [
    "calorie restrict", "fasting for weight", "cheat meal",
    "guilt free", "clean eating", "detox", "purge",
]

def is_safety_concern(query: str) -> bool:
    return any(term in query.lower() for term in SAFETY_TERMS)
```

Behavior: Decline without shaming. Offer an alternative path.

```
"I'm not able to help with that specific request. If you're looking for balanced
        meal ideas, I'm here for that."
```

Consent Refusal

Trigger: Request involves data the agent doesn't have permission to access or share.

```
def requires_consent(action: str, user_consents: set) -> bool:
    required = CONSENT_MAP.get(action)
    if required and required not in user_consents:
        return True
    return False
```

Behavior: Explain what consent is needed and how to grant it.

Integrity Refusal

Trigger: Request asks the agent to fabricate, deceive, or misrepresent.

Behavior: State what the agent knows and what it doesn't. Never fill gaps with invention.

Energy Refusal

Trigger: Request would consume disproportionate resources for marginal value.

Behavior: Suggest a more efficient approach or decline the specific implementation.

Legal Refusal

Trigger: Request would violate applicable regulations (GDPR, COPPA, EU AI Act).

Behavior: Cite the specific regulation and decline. Log the attempt.

Chapter 7: Drift Detection

Setting Baselines

Before you can detect drift, you need to know what "normal" looks like.

Create a baseline document:

```
 # Agent Behavioral Baseline
## Date: [YYYY-MM-DD]
## Agent: [name/id]

### Voice: How does this agent communicate?
- Tone: [formal/casual/technical/warm]
- Length: [concise/detailed/adaptive]
- Signature patterns: [specific phrases, formatting habits]

### Principles: What does this agent consistently do?
- Always: [list behaviors]
- Never: [list refusals]

### Performance: What are normal metrics?
- Response time: [range]
- Refusal rate: [percentage]
- Error rate: [percentage]
```

Measurement Cadence

Frequency	What to measure	Who measures
Weekly	Run governance scan, compare to last scan	Automated (CI)

Frequency	What to measure	Who measures
Monthly	Score all five drift dimensions (1-10 scale)	Team lead
Quarterly	Full behavioral audit against baseline	External reviewer

Escalation Thresholds

- Any single dimension exceeds 8/10: investigate immediately
- Composite change exceeds 2.0 from previous review: investigate cause
- Governance scan drops below previous score: block deployment until resolved

Adding the Scan to CI

```yaml
# .github/workflows/governance.yml
name: Governance Check
on: [push, pull_request]
jobs:
  governance:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npx agent-governance-check --json > governance-report.json
      - run: |
          SCORE=$(node -e "const r=require('./governance-report.json');
        console.log(r.score.found)")
          if [ "$SCORE" -lt "3" ]; then
            echo "Governance score below threshold: $SCORE/5"
            exit 1
          fi
```

Chapter 8: EU AI Act Mapping

The EU AI Act takes effect August 2, 2026. Here is how each governance property maps to specific requirements.

Governance Property	EU AI Act Article	What It Satisfies
Persistent Identity	Article 50 - Transparency	AI systems must disclose capabilities, limitations, and intended purpose
Refusal Capability	Article 14 - Human Oversight	Systems must support human intervention and override mechanisms
Decision Memory	Article 12 - Record-Keeping	Technical documentation must enable monitoring of AI system operation
Disagreement Protocol	Article 72 - Governance	Providers must establish AI governance frameworks
Drift Detection	Article 9 - Risk Management	Ongoing monitoring and risk management throughout the AI system lifecycle

This is not legal advice. Consult a qualified legal professional for compliance guidance specific to your jurisdiction and use case.

Chapter 9: Verify Your Work

After implementing governance, run the scan again:

```
npx agent-governance-check
```

Compare your before and after:

Property	Before	After	What Changed
Decision Memory	MISSING	FOUND	Added decision-log.jsonl + logging callback
Refusal Capability	MISSING	FOUND	Created restraint-spec.md + tool-level checks
Persistent Identity	PARTIAL	FOUND	Moved from system prompt to agent-charter.md
Disagreement Protocol	MISSING	FOUND	Added deliberation-framework.md
Drift Detection	MISSING	FOUND	Created baseline + CI governance check

Your score going from 1/5 to 5/5 is not a cosmetic improvement. It means your agent system has accountability, boundaries, memory, conflict resolution, and monitoring. It means you can answer the question: "How do you know your agent is governed?"

Next Steps

If you want...	Go to...
The complete trust architecture	Trust Architecture Complete ($149)
Hands-on governance audit	Book a discovery call
To contribute to the scanner	GitHub

Built by Evoked. evoked.dev

"We evoke - we never extract."